

UNIVERSIDAD LA SALLE



Facultad de Ingenierías y Arquitectura
Escuela Profesional de Ingeniería de Software

Curso: Calidad de Software
Docente: Maribel Molina Barriga

PRÁCTICA DE LABORATORIO 02

Cuestionario resuelto

Proyecto OLA — Observatorio Litoral de
Anomalías térmicas

*Respuestas al cuestionario de la guía, sustentadas con las mediciones y
evidencias del informe técnico de la práctica*

Integrantes

Frederick Mares Graos
Jhordan Huamani Huamani
Jorge Ortiz Castañeda
Miguel Angel Flores Leon
Piero Adrian Delgado Chipana

Arequipa, octubre de 2026

Las respuestas se apoyan en el módulo «Alertas y avisos» del backend de OLA, evaluado en el commit d5c054f de la rama main, y en las evidencias **E-01** a **E-11** del informe técnico (CS_LAB02_Informe_OLA.pdf). Las cifras provienen de las mediciones del 06/10/2026, cuyas salidas en bruto están en la carpeta evidencias/.

1. ¿Por qué bajo acoplamiento no significa ausencia de relaciones?

Porque un sistema solo produce valor cuando sus partes colaboran. Bajo acoplamiento significa que las relaciones son pocas, explícitas y dirigidas hacia elementos estables, de modo que un cambio en una parte no obliga a cambiar las demás.

El módulo evaluado ofrece los dos casos (**E-03**, **E-08**). `clock` no tenía ninguna relación ($Ca = 0$, $Ce = 0$) y eso era un defecto (H-04): una abstracción que nadie usa no desacopla nada, y el servicio seguía llamando a `datetime.now(UTC)` en tres lugares. Tras el rediseño, `clock` pasa a $Ca = 2$. En cambio, `notification_service` depende ahora de más módulos internos que antes (8 en lugar de 5) y aun así está menos acoplado en el sentido que importa: ya no construye sentencias SQL ni lee el reloj del sistema, sino que delega en el repositorio, en el despachador y en la abstracción `Clock`. Lo que se mide no es cuántas flechas hay, sino hacia qué apuntan y cuánto habría que cambiar si el otro extremo cambia.

2. ¿Qué atributo mejora con inyección de dependencias?

Sobre todo la comprobabilidad: si una clase recibe sus colaboradores en lugar de crearlos, en una prueba pueden sustituirse por dobles. También mejora la modificabilidad, porque una implementación puede reemplazarse sin tocar a quien la usa.

En el proyecto, `NotificationDispatcher` recibe por su constructor el almacén de avisos, el registro de canales y el reloj, definidos como protocolos. Gracias a eso sus 12 pruebas unitarias usan un almacén en memoria, un canal que falla a voluntad y un reloj fijo, y se ejecutan en 0.02 s sin PostgreSQL (**E-09**). Antes del cambio, la misma lógica de envío no tenía pruebas unitarias: solo podía comprobarse con las 34 pruebas de integración de `test_notifications.py`, que tardan entre 12.5 y 12.9 s en cinco ejecuciones porque necesitan la base de datos.

3. ¿Cómo se demuestra una mejora en modificabilidad?

Con un escenario de cambio concreto, implementado sobre los dos diseños con la misma especificación y medido con los mismos indicadores: módulos que hay que modificar, líneas de código existente que se reescriben, complejidad que se añade y pruebas que dejan de pasar. No basta con afirmar que el diseño «es más flexible».

En esta práctica se añadió el mismo canal nuevo, una pasarela por *webhook* siempre activa, sobre ambos diseños (sección 5 del informe, **E-10**). Los módulos existentes modificados bajaron de 6 a 4 y las líneas reescritas, de 13 a 4. El servicio, el repositorio y el router no se tocaron, y la complejidad del bucle de envío se mantuvo en 4, mientras que en el diseño actual subió a 6. La medición también muestra lo que no mejoró: las pruebas que dejan de pasar no disminuyeron (11 en el diseño actual y 12 en el propuesto), porque cuentan avisos y resúmenes que cambian por especificación. La evidencia son los parches de `escenario_webhook/`, la salida de `git diff --numstat` y la ejecución de la suite sobre cada implementación.

4. ¿Cuándo una clase grande puede estar justificada?

Cuando es grande pero cohesiva: tiene una sola razón para cambiar, baja complejidad, y dividirla solo repartiría el mismo concepto en varios archivos. El tamaño es una señal para revisar, no un defecto en sí mismo.

En OLA, `db/models.py` es el módulo más grande del alcance (229 líneas, 15 clases) y el más referenciado ($Ca = 20$), pero es declarativo: describe tablas y su complejidad máxima es 1 (**E-03**). Otro caso es `Settings`, con 21 campos que representan una única cosa, la configuración de la aplicación. En cambio, `notification_service`, con 141 líneas, sí debía dividirse: no por su tamaño, sino porque cambiaba por cinco motivos distintos (planificar, persistir, redactar, enviar y marcar la lectura de los avisos; H-01).

5. ¿Qué debe contener una evidencia auditable?

Lo necesario para que otra persona pueda localizarla, entenderla y reproducirla:

- un identificador único y la afirmación que respalda;
- la versión exacta del artefacto evaluado (aquí, el commit `d5c054f`);
- la herramienta, su versión y el comando o procedimiento usado;
- la fecha y el entorno de ejecución, incluida la configuración que puede afectar el resultado;
- el resultado en bruto, o un archivo legible que lo contenga;
- quién la obtuvo y sus limitaciones.

En el informe cada evidencia, de **E-01** a **E-11**, está numerada, se cita donde se usa y remite a un archivo entregado; las salidas de `pytest`, `ruff`, `mypy`, cobertura, tiempos y métricas están en evidencias/ con el identificador en su nombre. Las herramientas, sus versiones y la configuración del entorno están en la sección 1.2 del informe; el registro de evidencias, en la 6.2, y las limitaciones, en la 6.3.